

Université Assane Seck de Ziguinchor
UFR des Sciences et Technologies
Département d'Informatique

Année universitaire 2023-2024

Master 1 en Informatique, option GL
Unité d'Enseignement : Systèmes à base de connaissances
Élément Constitutif : Programmation fonctionnelle

Responsable CM / TD : Mouhamadou GAYE

Projet

Déroulement

Ce projet se déroule **du 30 juillet au 09 août 2025**. Le travail demandé se fait individuellement.

Enoncé

Dans ce mini-projet, on veut implémenter une structure de données qui s'appelle filtre de Bloom (chercher sur google de quoi il s'agit). Cette structure de donnée permet de tester de façon très efficace l'appartenance d'un objet dans un ensemble. Cependant, ce test est un test probabiliste qui peut donner des faux-positifs, c'est à dire qu'il y a une probabilité pour qu'il réponde oui alors que l'objet n'y est pas. En revanche, s'il répond non, c'est sûr à tous les coups que l'objet n'est pas présent. L'algorithme gère un tableau de bits en mémoire. Au départ, le tableau de bits est initialisé à 0. Quand on ajoute un élément à l'ensemble, l'algorithme va mettre k (k sera déterminé plus tard) bits du tableau à 1. Quand on voudra tester si l'élément appartient à l'ensemble, il suffira de tester si k bits ont été mis à 1. Pour déterminer quels bits sont mis à 1, on va utiliser une fonction de hachage. Une fonction de hachage prend en entrée un objet de type α et retourne un entier. La valeur retournée peut être vue comme une empreinte de l'objet initial. Évidemment, si cette fonction de hachage est implémentée comme suit :

$$f(x)=1$$

alors tous les objets sont identifiés de la même façon et la fonction de hachage est inutile. On préférera des fonctions compliquées qui garantissent deux entiers différents pour deux objets très similaires. Par exemple en utilisant la fonction `Hashtbl.hash` de la librairie standard, on peut observer les entiers générés pour les chaînes "a" et "aa" (ou "a" et "b").

Travail demandé

1. A partir du nombre de mots estimés dans l'ensemble ainsi qu'une probabilité p de faux-positif, l'article wikipedia permet de calculer m , la taille du tableau de bits ainsi que k le nombre de fonctions de hachage.

$$m = -\frac{n \times \log p}{(\log 2)^2}$$

$$k = \frac{m}{n} \times \log 2$$

Implémenter ces formules en Ocaml (m et k seront des entiers). Pour représenter un tableau de bit, on ne va pas utiliser le type `int array` ce qui serait terriblement peu efficace. Cependant, on peut astucieusement utiliser un `int` Ocaml pour représenter 31 bits (le type `int` d'Ocaml est stocké sur 31 bits). 31 bits c'est un peu embêtant à manipuler, et donc utiliser la librairie `Int32`.

2. Donner un type `bloom` pour la structure de donnée du filtre de Bloom. Ainsi que deux fonctions :

val get : bloom -> int -> bool qui retourne vrai si le $n^{\text{ième}}$ bit a été mis à 1 et faux sinon.

val set : bloom -> int -> unit qui met à vrai le $n^{\text{ième}}$ bit.

3. Implémenter maintenant les deux fonctions suivantes :

val add : bloom -> 'a -> unit qui ajoute un élément au filtre de bloom

val check : bloom -> 'a -> bool qui teste si un mot appartient au filtre de bloom

4. Télécharger le fichier `dictionary.txt` à l'adresse suivante :

<https://lsv.ens-paris-saclay.fr/~fthire/teaching/2016-2017/programmation-1/1/blooming/dictionary.txt>

Tester le filtre de Bloom avec ce fichier.

5. Comparer l'efficacité du filtre vis à vis d'un algorithme de recherche classique comme la recherche dichotomique.

Documents à rendre

Un dossier compressé à envoyer à l'adresse m.gaye@univ-zig.sn avant le **09 août 2025 à 23h59** comprenant :

- les codes sources ;
- des captures d'écran d'exemples d'exécution.